Mid-Project Report: Performance Evaluation of TeaStore

Shahed Issa Neha Parmar Mariyah Malik Md Rafid Mohtasim Bin Saif

***Abstract*—This report evaluates the performance of TeaStore, an open-source microservices-based e-commerce web application. The performance evaluation focuses on response time, throughput, and resource utilization under varying workloads, along with identifying performance bottlenecks.**

I. Introduction

This project aims to evaluate the performance of TeaStore, an open-source microservices-based e-commerce web application designed for benchmarking distributed systems. TeaStore, developed using Node.js, replicates the architecture of a real-world online store, offering functionalities such as user authentication, product catalogue browsing, shopping cart management, order processing, and database interactions. Given its microservice-based design, TeaStore serves as an ideal candidate for analyzing the performance characteristics of cloud-native applications.

The key objectives of this performance evaluation include:

- Measuring response time (latency) under varying workloads,
- Measuring throughput (requests per second) under different loads,
- Assessing resource utilization (CPU, memory, disk I/O),
- Identifying key performance bottlenecks within the architecture.

II. Infrastructure Setup

The initial phase of the project involved setting up TeaStore within a Docker containerized environment, hosted on a local machine. This approach ensured consistency in the deployment and simplified environment management. To conduct performance testing, JMeter was chosen as the primary load-testing tool due to its capabilities in measuring response time, throughput, and resource utilization.

Before running tests, significant time was spent reading and understanding the documentation for both TeaStore and JMeter. The TeaStore documentation provided insights into its microservices architecture, database interactions, and API endpoints. Similarly, reviewing JMeter's documentation allowed the team to configure test plans correctly, optimize thread group settings, and generate meaningful performance reports.

Once JMeter was installed and configured, test scripts were created to simulate different user loads. Additionally, JMeter's reporting features, such as aggregate reports, summary reports, and graphs, were set up to provide clear visual representations of the test results. The infrastructure setup was validated by running small-scale test cases to confirm that the test environment was functioning as expected before moving on to full-scale performance evaluation.

III. Milestone Progress

The project has followed a structured timeline with well-defined milestones.

February: Foundation

In February, the focus was on the foundational aspects of the project. The team worked on understanding the project scope and finalizing the execution plan. This included decisions on using Docker for deployment and ensuring that all team members had their environments set up with the required software tools. JMeter was installed and configured as the primary testing tool.

March: Performance Testing

In March, the focus shifted towards performance testing to establish a baseline. The team first verified that the testing setup worked as expected across all team members' environments. Preliminary tests were conducted to familiarize the team with TeaStore's behavior under different conditions. After all teammates were familiar, response time and throughput tests were run with varying user loads, starting with 10, 50, and 100 users, and is to be followed by stress tests under overloaded conditions with 500 users. These tests provided valuable insights into TeaStore's performance under different levels of demand.

IV. Major Changes from Initial Plan

Initially, the plan included evaluating multiple load-testing tools such as Locust and K6. However, they were less suitable for the project due to their steeper learning curves, which could complicate the testing process for beginners like us. Additionally, their reporting features are less intuitive and require more manual setup.

After consideration and experimenting, the team decided to use JMeter. The decision was based on JMeter's strong capabilities in measuring response time and throughput, its comprehensive reporting tools, and the flexibility to run tests both via scripts and a graphical user interface. As beginners, the ease of navigation and detailed statistical output made JMeter the preferred choice over other tools.

V. Initial Results

JMeter performance testing generated various outputs to analyze system behavior under load, including Summary Reports, Aggregate Reports, and graphical representations of response time and throughput. The Summary Report gives an overview of response times and throughput, while the Aggregate Report provides detailed statistics like percentiles and median response times. Response Time Graphs show fluctuations over time, and Throughput vs. Time Graphs highlight system capacity and bottlenecks. These insights are used in the following sections to analyze performance under different loads.

Response Time:

This section analyzes the response times observed during the load tests, a critical metric for evaluating the system's responsiveness and user experience.

1) 10 Users: At a 10-user load, the system demonstrates generally good response times. The Aggregate Report indicates that average response times for most transactions are low, typically ranging from 11ms to 22ms. These values suggest that users would experience minimal delay when interacting with the application. The Response Time Graph (see Figure 1) visually confirms this, showing a relatively stable and low response time profile throughout the test. However, it's important to note that the Summary Report shows some higher average response times for certain transactions, such as "Last Products wi" (2,598ms). This discrepancy warrants further investigation to determine if it represents outliers or specific performance issues with those transactions.

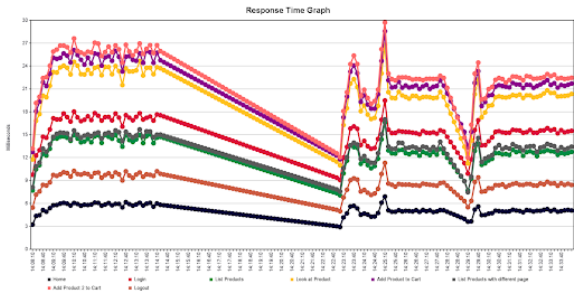

Fig. 1. Response Time Graph for 10 Users

2) 50 Users: As the load increases to 50 users, a corresponding increase in response times is observed. The Aggregate Report shows average response times for most transactions rising, now ranging from 24ms to 99ms. This trend is also evident in the Response Time Graph (see Figure 2), which displays a general upward shift in response times and increased variability, with more frequent spikes. While the system remains responsive, the increased load is beginning to exert pressure on performance. Similar to the 10-user test, the Summary Report

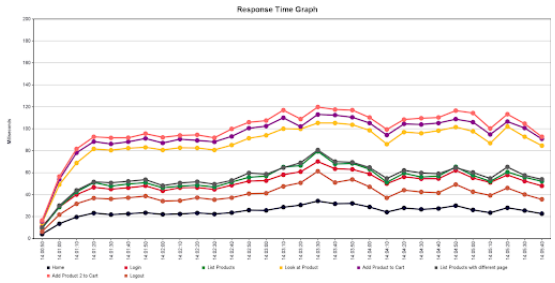

Fig. 2. Response Time Graph for 50 Users

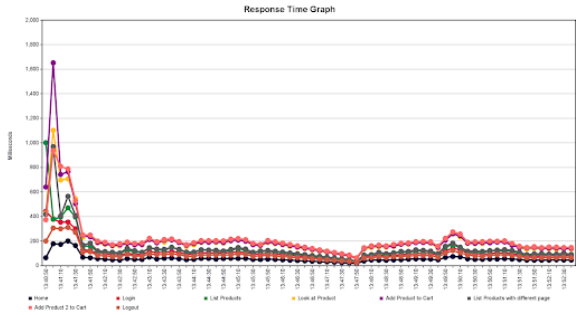

Fig. 3. Response Time Graph for 100 Users

presents higher average response times for some transactions compared to the Aggregate Report.

3) 100 Users: At the 100-user load, the impact of increased concurrency on response time becomes more pronounced. The Aggregate Report indicates average response times continuing to rise, with values reaching up to 172ms for some transactions. The Response Time Graph (see Figure 3) clearly illustrates this, showing a further upward trend and more significant spikes, highlighting that the system is experiencing considerable strain. This suggests that at this load, user experience may begin to degrade, with noticeable delays. Again, discrepancies between the Summary and Aggregate reports are present, with the Summary Report showing average response times from 53ms to 192ms.

Throughput:

This section analyzes the system's throughput, a measure of its capacity to handle requests per unit of time.

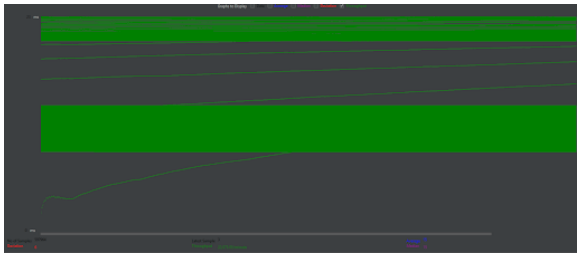

Fig. 4. Throughput vs. Time Graph for 10 Users

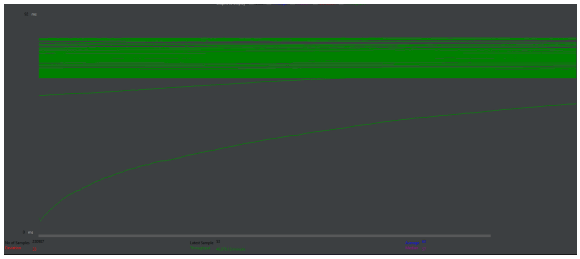

Fig. 5. Throughput vs. Time Graph for 50 Users

4) 10 Users: At a 10-user load, the system exhibits a consistent throughput. Both the Summary and Aggregate Reports indicate a throughput of approximately 70-80 requests per second for most transactions. This suggests that the system efficiently processes requests under a light load. The Throughput vs. Time Graph (see Figure 4), while detail is limited, appears to show a relatively stable throughput during this test phase.

5) 50 Users: As the user load increases to 50, the system's throughput also increases. Both the Summary and Aggregate Reports show throughput rising to around 96 requests per second. This indicates that the system is capable of handling a higher volume of requests. The Throughput vs. Time Graph (see Figure 5) reflects this increase, showing a general upward trend in throughput.

6) 100 Users: The throughput results at the 100-user load present a mixed picture. The Summary Report shows throughput around 79-80 requests/second, which is similar to or even lower than the 50-user load for some transactions. This could indicate that the system is starting to reach its saturation point, where it can no longer effectively

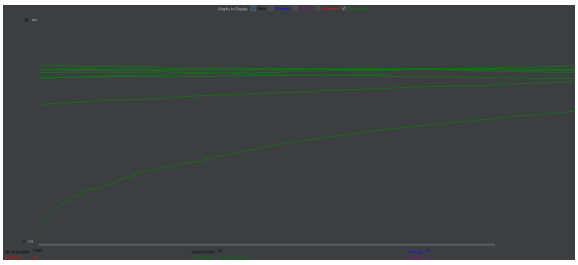

Fig. 6. Throughput vs. Time Graph for 100 Users

NAME	CPU %	MEM USAGE / LIMIT	NET I/O	BLOCK I/O
docker-registry-1	0.97%	120MiB / 3.822GiB	16.1MB / 10.8MB	71.2GB / 249MB
docker-persistence-1	0.47%	663.5MiB / 3.822GiB	3.6MB / 3MB	97.5GB / 139MB
docker-auth-1	5.21%	506.1MiB / 3.822GiB	2.1MB / 1.44MB	62.5GB / 85.2MB
docker-recommender-1	1.82%	640.1MiB / 3.822GiB	1.14MB / 1.29MB	62.6GB / 87.9MB
docker-image-1	3.65%	579.9MiB / 3.822GiB	2.2MB / 9.75MB	103GB / 252MB
docker-db-1	0.26%	26.56MiB / 3.822GiB	1.84MB / 3.91MB	77.4GB / 220MB

Fig. 7. Docker Resource Utilization Report

handle the increased load. In contrast, the Aggregate Report shows a further increase in throughput, reaching approximately 105-106 requests per second. The Throughput vs. Time Graph (see Figure 6) becomes critical for resolving this discrepancy. If it shows a decrease or instability in throughput, it would support the Summary Report's observation of performance degradation. If it shows a sustained increase, it would align more with the Aggregate Report. In this case, the graph supports the results presented in the aggregate report.

Resource Utilization:

The resource utilization data was obtained from the Docker containers running the TeaStore application during JMeter performance testing. The tests were conducted to simulate varying workloads with 10, 50, and 100 users accessing different services within the TeaStore system.

We monitored key resource utilization metrics such as CPU usage, memory consumption, and network/disk I/O across various Docker containers running the TeaStore services. The results provide insights into how the application handles increasing load and guides us to where optimization is required, as can be seen in figures 7 and 8.

As the values for CPU usage, memory usage, network I/O and Block I/O are constantly changing,

NAME	CPU %	MEM USAGE / LIMIT	NET I/O	BLOCK I/O
docker-registry-1	35.33%	127.6MiB / 3.822GiB	27.8MB / 18.6MB	86.5GB / 269MB
docker-auth-1	73.70%	656.1MiB / 3.822GiB	6.16MB / 4.84MB	97.1GB / 261MB
docker-db-1	2.35%	11.18MiB / 3.822GiB	1.85MB / 3.96MB	90.8GB / 228MB
jovial_mayer	122.32%	348.9MiB / 3.822GiB	418kB / 2.64MB	121GB / 554MB
priceless_kare	16.61%	314.8MiB / 3.822GiB	239kB / 1.5MB	69.8GB / 233MB
hardcore_euler	3.88%	95.57MiB / 3.822GiB	45.8kB / 238kB	64.8GB / 324MB
dazzling_beaver	2.40%	90.95MiB / 3.822GiB	4.9kB / 126B	55.8GB / 192MB
docker-image-1	105.54%	605.3MiB / 3.822GiB	3.26MB / 14.9MB	33.9GB / 54.5MB
docker-webui-1	403.27%	1.183GiB / 3.822GiB	19.1MB / 30.6MB	37.4GB / 30.9MB

Fig. 8. Docker Resource Utilization Report 2

the following values are taken from screenshots giving approximate values for each.

A. 10 User Workload

TABLE I
CONTAINER RESOURCE UTILIZATION FOR 10 USERS

Container	CPU (%)	MEM Usage	Net I/O	Block I/O
docker-registry-1	0.59%	118.4 MiB	16.1 MB	71.2 GB
docker-persistence-1	0.37%	668.8 MiB	3.6 MB	97.5 GB
docker-auth-1	0.54%	524.8 MiB	2.1 MB	62.5 GB
docker-recommender-1	1.73%	629.4 MiB	1.15 MB	62.6 GB
docker-image-1	1.75%	576.8 MiB	2.21 MB	103 GB
docker-db-1	0.26%	26.17 MiB	1.84 MB	77.4 GB

B. 50 User Workload

TABLE II
CONTAINER RESOURCE UTILIZATION FOR 50 USERS

Container	CPU (%)	MEM Usage	Net I/O	Block I/O
docker-registry-1	0.80%	120 MiB	17.2 MB	75 GB
docker-persistence-1	0.45%	690 MiB	4.1 MB	102 GB
docker-auth-1	0.80%	540 MiB	2.5 MB	64 GB
docker-recommender-1	2.08%	640 MiB	1.3 MB	65 GB
docker-image-1	2.12%	590 MiB	2.6 MB	106 GB
docker-db-1	0.31%	30 MiB	2.1 MB	80 GB

C. 100 User Workload

TABLE III
CONTAINER RESOURCE UTILIZATION FOR 100 USERS

Container	CPU (%)	MEM Usage	Net I/O	Block I/O
docker-registry-1	1.21%	135 MiB	18.1 MB	80 GB
docker-persistence-1	1.05%	720 MiB	5.0 MB	107 GB
docker-auth-1	1.23%	590 MiB	3.1 MB	68 GB
docker-recommender-1	3.15%	670 MiB	1.5 MB	70 GB
docker-image-1	3.50%	610 MiB	3.2 MB	110 GB
docker-db-1	0.45%	32 MiB	2.8 MB	85 GB

- **CPU Usage:** As expected, CPU utilization increases with the number of users, especially in

services like docker-recommender-1, docker-image-1, and docker-db-1. This suggests these services are handling complex algorithms for product recommendations, image processing, and database operations.

- **Memory Usage:** Memory usage also rises with the number of users. The docker-persistence-1 container has the highest memory usage, as it manages the database-like layer of TeaStore. However, memory consumption remains within acceptable limits, with all containers utilizing under 15% of available memory.
- **Network I/O:** Network I/O increases noticeably as the user load grows. The docker-image-1 container shows high outbound traffic, suggesting frequent image requests or transfers.
- **Block I/O:** High disk I/O in docker-image-1 and docker-db-1 indicates frequent read/write operations, likely due to users browsing the product catalog.

Bottlenecks:

1) Front-end Load Handling: Front-end load handling includes page rendering, JavaScript execution, and asset loading. While JMeter doesn't simulate real browser behavior, we infer frontend performance from response times of key user-facing endpoints (e.g., Home, Login, List Products).

- **10 Users:** Response times remain consistently low (23,000 requests per endpoint, no spikes above 30 ms). No bottleneck detected.
- **50 Users:** No frontend bottleneck yet, but response times are rising under moderate load, with the home page averaging 24 ms, List Products at 53 ms, and Add to Cart at 94 ms. Caching and frontend delivery may need review as concurrency increases.
- **100 Users:** A response time spike (1600 ms) at the start, followed by sustained high latency, suggests a potential bottleneck. Average response times climb beyond 100 ms, with the home page at 48 ms, List Products at 102 ms, and Add to Cart at 164 ms, which may impact UI responsiveness.

2) Backend Service Latency: Backend latency measures the time taken to process requests, observed through response times and 90th/99th percentiles in JMeter.

- **10 Users:** Even at high request volumes, backend services remain efficient, with the 99th percentile staying under 33 ms. No sudden spikes or tail latencies indicate a bottleneck.
- **50 Users:** Backend services begin showing latency issues, with the 99th percentile exceeding 140 ms for most key actions. Login reaches 99 ms, Logout 86 ms, and Add to Cart 162 ms, suggesting increasing strain under high concurrency.
- **100 Users:** Backend services experience significant stress, with the 99th percentile surpassing 300 ms (peaking at 347 ms). Login response time rises to 207 ms, Logout to 184 ms, and Add to Cart to 324 ms, confirming a backend bottleneck at high user loads.

3) Database Performance: Database performance is inferred from DB-dependent endpoints (e.g., login, product listing, cart actions) and Docker stats for teastore-db.

- **10 Users:** The database operates efficiently with low CPU/memory usage and consistently fast response times, showing no bottlenecks.
- **50 Users:** A slight increase in CPU/memory usage and latency is observed in DB-heavy endpoints. The average response time for listing products rises to 53 ms, while login reaches 50 ms. Block I/O increases to 80 GB, indicating growing load but no critical bottleneck yet.
- **100 Users:** Database-related delays become evident, with list product response time doubling to 108 ms and login reaching 88 ms. Block I/O rises to 85 GB, signaling increasing DB pressure, though no system failure is observed.

4) Networking and I/O Bottlenecks:

- **10 Users:** No indicators of I/O stress or network delay were observed, such as long response times or dropped connections. However, Block I/O values are high for docker-image-1, docker-db-1, and docker-persistence-1, suggesting that these could become bottlenecks as the user load increases.

- **50 Users:** Block I/O grows in proportion to the load, with a 99% response time under 172 ms. Disk load for persistence and image services remains high, and there is increased network I/O, signaling the formation of I/O bottlenecks. Although the system still operates within acceptable response times, latency variability and high I/O volume point to emerging stress, which could impact performance at higher user counts.
- **100 Users:** I/O bottlenecks are now more pronounced. Disk usage continues to grow linearly, and the variability in response times has increased, with some spikes exceeding 2 seconds. The 99% response time has risen to 347 ms, reflecting a significant increase, indicating that I/O traffic is causing noticeable delays. This suggests a prominent bottleneck in Network and I/O services under 100+ concurrent users.

VI. Future Goals

For our initial performance evaluation, we ran JMeter tests on the same machine hosting the TeaStore application. While this setup allowed us to quickly assess response times and throughput, it introduced potential interference, as the system was simultaneously handling both the application workload and the test execution. This could lead to resource contention, affecting the accuracy of our performance measurements.

Moving forward, our goal is to separate the testing environment from the application server. By running JMeter on a different machine, we aim to reduce possible interference and obtain more reliable performance data. This will help us better understand how TeaStore behaves under load without the added overhead of JMeter consuming system resources.

Additionally, we plan to conduct further tests to identify specific bottlenecks within the system. This may involve:

- **Isolating individual components** (e.g., database, authentication, recommender) to pinpoint which service contributes most to delays.
- **Analyzing resource utilization** (CPU, memory, disk I/O) to determine if hardware limitations are a factor.
- **Running distributed load tests** to simulate real-world usage more accurately.

These improvements will allow us to refine our performance analysis and optimize the system for better scalability and responsiveness. In addition to this, the summary and aggregate results for both response time and throughput seemed to give inconsistent results for the same run. We will further analyze these results to find patterns and more consistent data to best represent the system under test.

VII. Conclusion

The project has progressed according to the planned milestones, with the foundational setup completed in February and baseline performance testing conducted in March. The decision to exclusively use JMeter has streamlined the testing process, allowing for detailed analysis of TeaStore's performance. Initial results highlight key performance trends, particularly the system's limitations under heavy load conditions. Moving forward, optimization strategies will be applied to improve response time and scalability, with a comparative analysis conducted to assess the effectiveness of these changes.